CAPRM-Flood: A Reproducible C++/Python Framework for Property-Level Flood Exposure Indexing

Sterling Alexander
Department of Computer Science
Golisano College of Computing and Information Sciences
Rochester Institute of Technology
Rochester, NY 14623
sa5836@rit.edu

***Abstract*—CAPRM-Flood is a reproducible C++/Python pipeline that extracts property-level flood-hazard evidence from public geospatial data and combines that evidence into a relative exposure index for all 267,362 properties in Monroe County, New York. Each performance-relevant spatial kernel is implemented independently in C++ and validated field by field against a trusted Python reference. Five index designs were evaluated for the nearest-water computation: brute force, a feature-level bounding volume hierarchy, a segment-level bounding volume hierarchy, and Hilbert-ordered search seeded by binary search and by a learned recursive model index. All five reproduce the reference on all ten fields of the nearest-water evidence record for all 267,362 properties, with a maximum absolute distance error of 9.16×10^{-10} m. Measured wall-clock time does not follow the ordering predicted by the analytic probe metric reported in the learned-index literature. The segment-level hierarchy is the fastest design at 34.1 microseconds per property, while both one-dimensional designs are approximately twice as slow, including the design that reduces the probe metric by a factor of 3.20. A decomposition of query time attributes this result to search cost rather than to verification cost. A coordinate-only neural surrogate for the exposure index was subsequently evaluated under a buffered blocked spatial split whose block geometry is justified against a correlation length estimated from the workload; under that split the surrogate is not separable from a constant predictor, and a prediction registered before training is refuted. The exposure index is a relative regional ranking and is not a flood probability, an expected loss, or an actuarial estimate.**

***Index Terms*—spatial indexing, bounding volume hierarchy, space-filling curves, learned index structures, geospatial data processing, spatial cross-validation, reproducibility, flood exposure**

I. Introduction

Flooding is the most common weather-related natural disaster [1]. In the northeastern United States, climate change is increasing exposure to flooding, partly through increases in heavy precipitation [2]. Flooding can damage buildings, homes, roadways and water and electrical infrastructure [2]. FEMA estimates that just one inch of flooding can cost up to approximately $25,000 in damage to a home [3]. Monroe County has been included in 4 FEMA flood-related disaster or emergency declarations from 1956–2022, the most recent of which took place in 2017; further, the Monroe County hazard plan identifies multiple local mechanisms increasing flood susceptibility, including lakeshore flooding and erosion along Lake Ontario, riverine flooding, urban drainage flooding, and flash flooding. FEMA reports that each dollar spent on mitigation saves two dollars in post-flood recovery costs [4].

Property-level flood analysis uses geospatial data detailing mapped flood hazards, terrain, hydrography, precipitation and property location to assess vulnerability and predict loss. FEMA maintains the National Flood Hazard Layer as a database containing geospatial data for flood hazard prediction, including both flood classification and level of risk [5]. The National Flood Insurance Program (NFIP) uses the FEMA Flood Map Service Center (MSC) as its official source for flood hazard information [6]. Beyond FEMA's MSC, the New York State GIS Program Office provides parcel-centroid data for all 62 New York counties; centroids are generated from the state's parcel-map program and are mapped to assessment-roll data [7]. When using multiple data sources like these simultaneously, construction of a property-level feature table requires the reconciliation of data sources comprising different schemas, geometry types, coordinate reference systems, and file formats.

A spatial join combines observations by geometric relationships instead of a shared identifier or key [8]. Across modern spatial analytics systems driven by spatial joins, research has observed significant variations in performance, demonstrating the need for explicit benchmarking over assumptions about equivalent performance [9]. Established libraries facilitate geospatial feature extraction: GeoPandas allows for planar spatial operations driven by predicates, such as `intersects` [10]. Because GeoPandas performs planar operations, its geometric and distance calculations depend on appropriate coordinate reference system selection. Shapely offers distinct topological predicates: `contains` excludes points that lie only on a polygon boundary, whereas `covers` includes geometries lying within either the interior or boundary of the selected geometry [11]. A common structure across indexed spatial processing systems is the filter-and-refine structure, which first searches bounding boxes and then applies a predicate to filter

candidate geometries [12]. Boundary point results may differ under different predicate semantics or across different spatial analytics libraries; in general, large batches benefit from spatial indexes that reduce the number of exact comparisons needed to identify matching geometry. These traits suggest the need for separate evaluations of performance and correctness in a spatial feature-extraction system.

County-scale property analysis also bears computational costs which are distinct from parcel-by-parcel analysis. Calculating nearest-mapped water feature distance for a property requires a comparison between that property and the individual segments composing each candidate feature; the naïve cost of this operation is the product of two large counts for a large study area containing many of both properties and features. The standard means of addressing this computational cost is via one of many contemporary spatial indexing strategies: hierarchical bounding structures, space-filling curve orderings, and learned index models leverage different, conflicting assumptions about the study region's underlying geometry. Literature on a learned-index approach to spatial joins reports gains in probe counts and model size rather than in measured time on a concrete workload. Investigating the probes-to-execution time relationship depends on the specific query, index and machine.

This project seeks to answer the following question: Can a reproducible C++/Python pipeline extract property-level flood-hazard features from public geospatial data while preserving exact agreement with a trusted Python baseline, and which spatial-indexing strategies (classical, learned, or approximate) actually reduce large-batch computation time under measured evaluation?

This paper presents CAPRM-Flood, a benchmarked spatial feature-extraction pipeline for property-to-hazard joins. The study region is Monroe County and Rochester, New York. The system takes as input a stable property identifier and coordinates and outputs traceable per-property evidence fields, normalized components, and a preliminary relative exposure index. The system comprises two layers of processing driven respectively by Python and C++. Python responsibilities include ingestion, schema cleaning, normalization, GeoPandas/Shapely reference computation, validation and reporting; C++ responsibilities include batch point-in-polygon processing with per-feature bounding box prefiltering and relevant flood-risk feature lookups, such as distance to the nearest mapped water source.

CAPRM-Flood presents a reproducible workflow for deriving traceable property-level features from public property and flood-hazard data. It provides a C++ spatial-processing core for batch property-to-hazard lookups and a validation framework for comparing its correctness and performance against GeoPandas and Shapely, including point-in-polygon agreement and boundary-case analysis. Finally, it provides a performance evaluation framework with distinct benchmarks for ingestion, index construction, computation time, throughput, and scaling. These components provide transparent screening and feature infrastructure for reproducible research and the preparation of validated data for later risk analysis or modeling.

In addition to the geospatial feature extraction framework, CAPRM-Flood contributes a five-rung indexing ladder for the nearest-water computation. Each rung preserves exactness in guaranteeing that the nearest-water feature retrieved is veritably so; each rung is also validated against the same frozen Python reference. By differing rungs only in index structure and seed strategy, measured differences are attributable to indexing rather than a specific declared CAPRM-Flood configuration. Finally, CAPRM-Flood explores the feasibility of a coordinate-based neural surrogate for the relative exposure index, computed under a buffered, blocked spatial split, with a predicted claim which was reported as refuted.

II. Related Work

Works related to CAPRM-Flood can be grouped according to function. As FEMA's database of current effective flood-hazard data, the NFHL provides flood-hazard geometries and regulatory mapping information derived from effective flood maps and Letters of Map Change (LOMC) [5]. FEMA's National Risk Index offers baseline risk measurements for flooding among 17 other natural hazards at the community level, and is accessible to the public through the Resilience Analysis and Planning Tool (RAPT) [13]. Additionally, FEMA provides Hazus as a standardized set of hazard-estimation tools and corresponding data; Monroe County's 2023 Hazard Mitigation Plan (HMP) utilizes Hazus to predict loss and damages due to flooding [4], [14]. Hazus flood-model methodology is implemented in FAST to swiftly analyze building-level flood risk [15]. These resources supply hazard information or perform broader risk analysis, whereas CAPRM-Flood investigates reproducible computations of property-level features with transparency in both feature derivation and performance evaluation.

Multi-Criteria Decision Analysis (MCDA) provides an established set of methodologies for assessing flood risk. In particular, the Analytic Hierarchy Process (AHP) facilitates the mapping of relative weights to different geospatial criteria [16]. GIS-based multi-criteria analysis has been used to map flood vulnerability in the trans-boundary Shatt Al Arab basin (Iraq-Iran) [17]. Factors used in MCDA studies include slope, elevation, land use, river proximity, drainage density, rainfall and soil type, while AHP facilitates a structured comparison of each criterion's influence on flood vulnerability [18]. CAPRM-Flood uses explicit component weights to produce a preliminary countywide exposure index, while separately evaluating implementation correctness and computational performance.

CAPRM-Flood evaluates correctness using GeoPandas and Shapely as a Python reference environment. GeoPandas provides predicate-driven spatial joins and relies on Shapely for performing geometric operations [10]. Existing tools facilitate spatial feature extraction in C++, though prior systems research demonstrates significant performance variation across spatial systems and workloads [9]. Boost.Geometry's R-tree

implementation and spatial query interface are feasible components of a C++ core for executing indexed spatial queries [19]. In leveraging these resources, CAPRM-Flood contributes a bounded case study involving property points and real FEMA polygon geometries.

Hierarchical and order-based indexing present as two distinct approaches to proximity search. Bounding volume hierarchies prune subtrees against a single lower bound, and the quality of this pruning depends on the fit of bounding volumes around their contained geometries. Space-filling curves reduce a range query to a small number of contiguous intervals by imposing an ordering on multidimensional data which places spatially proximate objects in proximal indices of a sorted one-dimensional array. Moon et al. demonstrate, in analysis of the Hilbert curve's clustering properties, that the number of contiguous clusters corresponding to a query region scales with surface area rather than volume, establishing an upper bound on how well such ordering can approximate spatial locality [21]. CAPRM-Flood implements both approaches over identical geometry at an identical configuration, and reports the measured difference in computation time and in work performed.

Learned index structures recast the spatial index as a model. Kraska et al. describe an index which predicts a key's position in a sorted array; this prediction replaces the comparison cascade of a search tree with an inference step and bounded local correction [20]. The main metrics used to observe performance gains from learned indexes are the number of probes necessary to locate a key in the sorted array, and the memory footprint of the structure itself. CAPRM-Flood implements a recursive model index over a Hilbert key array and reports the probe metric alongside measured wall-clock time for identical queries; the two measurements do not agree, and the corresponding performance evaluation of the approach analyzes this disagreement.

Model evaluation over spatially-dependent data requires a partitioning strategy which accounts for this dependence. Roberts et al. demonstrate that random cross-validation applied to data possessing spatial, temporal, hierarchical, or phylogenetic structure underestimates predictive error as a result of the interleaving between training and test data in coordinate space. Accordingly, they recommend blocking schemes with a block size selected based on the dependence structure of the data [22]. CAPRM-Flood follows this recommendation and additionally retains the random split as an explicitly-failing control to observe the magnitude of the error introduced by a random split of training and test data.

Networks accepting low-dimensional coordinates as input show spectral bias toward low-frequency functions. Tancik et al. demonstrate that mapping such coordinates through a set of random Fourier features prior to the network improves its capacity to represent high-frequency variation [23]. The surrogate evaluated by CAPRM-Flood uses this construction; however, the target field is not a high-frequency continuous variation but a field containing a discrete step introduced by categorical polygon membership. CAPRM-Flood characterizes this resulting failure mode rather than reporting an aggregate error metric.

III. System Architecture

CAPRM-Flood implements a staged geospatial evidence pipeline, with distinct stages for ingestion of public data, computation of results via a trusted reference methodology, independent spatial computation, results validation, evidence generation, and preliminary indexing. The architecture is structured to preserve auditable property-level evidence, validate independent computations, use coordinate reference system (CRS) normalization in support of the spatial operation performed, and separate evidence and scoring processes.

The architecture's first stage performs Python ingestion and subsequent caching: property points are loaded or retrieved, along with FEMA flood-hazard polygons, county-boundary geometry, hydrography features, and terrain raster data. For standardization purposes, raw data is normalized into standard schemas, source identifiers are recorded, operation-specific CRS values are validated across relevant data sources, and artifact manifests and checksums corresponding to the target dataset are cached for reproducibility and future data monitoring. In total, this architecture uses a Python environment to perform data access, CRS conversion, metadata sampling, reference calculations, results validation across software environments, and evidence presentation.

The dataset surveyed geographically spans Monroe County, using canonical property identifiers derived from NYS parcel centroid data. This workload represents each canonical property identifier as a unique row, facilitating the consolidation of all evidence features under the same per-property key.

The first property feature surveyed for CAPRM assessment is FEMA flood-zone membership including a Special Flood Hazard Area (SFHA) designation for each property point. A Python reference implementation collects data for this feature family by projecting property points and FEMA polygons into a shared CRS; a spatial membership predicate is subsequently applied via GeoPandas/Shapely. FEMA features referenced by this reference implementation are flattened into polygon parts and rings for a C++ point-in-polygon membership computation, including hole handling and feature-specific bounding box filtering. Evidence validated between code environments for each unique property point includes polygon matching verification, FEMA flood-zone membership label, SFHA flag, and source feature identifier and index.

CAPRM uses nearest-water distance as its second evidence family. The reference computation in Python projects both property points and US Geological Survey hydrography feature data to EPSG:26918 as a shared CRS, builds a spatial index, computes point-to-feature distances, and records feature and identification data corresponding to each point's selected nearest hydrography feature. C++ performs two operations over this same evidence family: first, a brute-force implementation of the methodology executed in Python; second, an indexed feature-level bounding-volume hierarchy. While both C++ implementations share geometry semantics, the

indexed search strategy prunes feature bounding box queries that cannot improve or tie the active result for nearest feature.

Terrain-derived evidence is used as the third risk assessment feature. Because this implementation layer depends on raster-window sampling and raster reprojection instead of vector geometry operations, it is currently implemented entirely in Python. A source digital elevation model (DEM) is reprojected to EPSG:26918 to enable usage of meter-based coordinates. At each property centroid, data is recorded for elevation, local mean elevation, relative elevation, slope in degrees, sampling radius, and terrain CRS. To preserve the traceability and independent analysis of the DEM data, the terrain evidence is recorded as a separate evidence table and manifest with a raster checksum separate from the integrated FEMA membership and hydrography evidence table.

Upon extraction of all evidence fields, CAPRM-Flood builds an exposure index table. In doing so, it converts the FEMA/hydrography and terrain evidence tables into normalized 0–100 component scores and applies explicit component weights to produce a countywide relative exposure index with percentile ranking. In total, a staged architecture is utilized in order to provide two forms of defensibility. Each evidence family can be inspected independently before scoring, and both implementation correctness and computational performance are calculated in Python as expected behavior and then compared against C++ outputs. Thus, property-level evidence and preliminary relative indexing are both supported with a clearly defined scope concerning claims made by the system.

A fourth architectural layer supports comparative indexing. Ingesting the same exported fixtures as used by the production water computation, this layer is validated against the same frozen Python reference; however, it writes to a separate output tree and is not included in evidence generation. In this way, the evidence table is produced by a single fixed pair of implementations regardless of which indexing experiments have been executed, and the recorded manifest embeds the benchmark summary corresponding to that pair. Keeping the two separate means the provenance of the evidence table does not depend upon experimental activity occurring alongside it.

IV. Implementation

CAPRM-Flood features a deterministic, reproducible implementation in C++/Python over Monroe County property workloads. Python components perform data retrieval, caching and cache validation, CRS reprojection, trusted reference computations, evidence consolidation and preservation, and overall pipeline coordination. C++ components perform a separate batch spatial computation over fixtures exported by Python as part of pipeline coordination. This allows Python geospatial libraries to maintain a reproducible reference behavior while developing C++ implementations, ultimately yielding a twice-evaluated pair of geometry operations and indexed-search performance whose results can be evaluated for procedural consistency.

A. Property Workload Construction

The property workload comprises all NYS parcel centroid records addressed specifically within Monroe County. Properties have identifying SBL values; these are used to deterministically build workloads by processing identifiers in ascending order, retaining the first non-missing occurrence of each canonical identifier, and discarding later duplicate identifiers. In total, the workload comprising Monroe County spans 267,362 unique property identifiers; individual workloads taken for development purposes contained subsets of Monroe County's data of sizes 1K, 10K, and 100K property points. To preserve reproducibility while testing scalability, all workloads were developed via computational fixtures versus random statistical sampling.

B. FEMA Flood Zone Evidence

During the evidence stage, each property centroid is marked based on whether the property lies within a FEMA flood-hazard polygon, and if so, associated flood-zone evidence is recorded. Property points and FEMA National Flood Hazard Layer (NFHL) polygons are reprojected to the shared FEMA topology CRS; a spatial join is subsequently performed to map the two layers together, and a property-level baseline is recorded for each point including property identifier, source and projected coordinates, flood-zone and SFHA membership labels, source geometry identifier, feature index, polygon-match status, and a simplified SFHA Boolean result.

The independent C++ FEMA stage ingests the reprojected property coordinates in addition to Python-flattened polygon-ring fixtures, both of which share a CRS upon ingestion. The polygon-ring table includes polygon parts, ring indices, vertex coordinates, flood-zone and SFHA labels, source geometry identification, and only the feature indices from Python which are necessary to perform a full comparison of computed results. Among all FEMA data extracted, only those FEMA features referenced by the Python baseline implementation are exported to C++ to minimize the functional export size. After ingestion, C++ reconstructs polygon parts and rings, applies bounding-box prefiltering for each FEMA feature, and excludes interior holes to compute property-specific evidence fields for matched polygon status, flood-zone and SFHA label, and feature index.

C. Nearest Water Evidence

Distance from each property's centroid to the nearest hydrography feature is computed using data from the United States Geological Survey 3D Hydrography Program cache. This cache, bounded by a 20 km buffer around official Monroe County coordinates, includes retained flowline and waterbody feature information. The CRS is maintained as EPSG:26918 to present distances in meters.

A reference implementation of nearest-water computations is performed in Python. This environment loads the hydrography cache, validates feature identifiers and geometry data, normalizes the CRS, constructs a spatial index via Shapely, and computes exact point-to-feature distances. Output records

nearest-water distance, along with the identifier, class and type of the selected nearest feature, the source identifiers, name and tie count corresponding to the selected feature, and CRS. Nearest-feature ties are resolved deterministically via canonical feature ID, i.e., selecting the first sorted feature for reported metadata fields while preserving tie count.

Python reference behavior is reproduced in two C++ implementations. First, a brute-force program considers each retained hydrography feature for each property when calculating its nearest feature, computing exact point-to-segment distances for line features and point-to-polygon distances for waterbody features. Points inside of waterbody polygons are considered to be zero meters from the nearest feature; polygon holes are excluded from waterbody interior treatment. The second C++ implementation preserves distance semantics while implementing a feature-level bounding-volume hierarchy over hydrography feature bounding boxes. In doing so, nodes and features are pruned which cannot improve or tie the current nearest hydrography feature.

D. Segment-Level Hierarchy and Entry-Extent Capping

The feature-level hierarchy bounds each water feature within a single box. For an extended feature such as a river reach, this box is large and predominantly empty; the bound it supplies is therefore weak, and the corresponding subtree is rarely rejected. CAPRM-Flood's third indexing strategy indexes at segment granularity rather than indexing features, reducing the average number of candidate features surviving the descent from 5.4976 to 1.4662 per property.

Long segments reintroduce the same condition at a smaller scale: An axis-aligned box drawn around a long diagonal segment is predominantly empty space, and the distance from a query point to that box can be substantially smaller than the distance to the segment contained within it. Accordingly, the implementation caps entry extent by splitting any segment exceeding a threshold length. This split is exactness-preserving by construction, because the minimum of a distance function over a partition of a set equals its minimum over the set; no computed distance changes as a result of splitting.

The cap value was selected against measured time rather than against minimized work. A 10 m cap produces fewer segment checks per property (9,125) than the 25 m cap (9,408) and is nevertheless slower, at 9.713 s against 9.596 s, because it inflates the index from 1,189,589 entries to 1,683,537 and the resident structure from approximately 120 MB to 157 MB. Work counted in comparisons and work paid for in cache misses are distinct quantities, and only the latter appears on a clock. The 25 m cap is subsequently held constant across every rung employing a cap, which is what permits the comparison between the segment-level hierarchy and the Hilbert ordering to be interpreted as a statement about dimensionality rather than about configuration.

Candidate verification admits two configurations, and results are reported under both because they differ substantially in cost. Under original verification, the features surviving the descent are checked against their complete unsplit geometry; under split verification, they are checked against the capped entries alone. Both produce identical distances, but they differ in the number of checks performed and in the cost of each check, so a comparison between index designs is meaningful only when the verification configuration is held fixed.

E. Hilbert-Ordered Search

The fourth rung replaces the hierarchy with a total order. Each of the 1,189,589 index entries is reduced to the midpoint of its segment, that midpoint is mapped to a single 64-bit Hilbert key at 32 bits per axis, and the resulting key array is sorted. Payload arrays holding actual segment endpoints and feature identifiers are permuted by the same permutation and thus remain aligned with the keys. At this resolution the 1,189,589 entries occupy 1,189,510 distinct cells, so the ordering is very nearly injective over the workload.

Sorting by Hilbert key does not sort by distance to an arbitrary query point, and no query-independent ordering could do so, because the ordering is fixed before any query exists. What the ordering supplies instead is spatial locality, which is the property permitting a two-dimensional disk to map to a small number of contiguous key intervals and therefore to a small number of contiguous array slices. That locality is imperfect, and its imperfection is measurable rather than theoretical.

Because keys are constructed from segment midpoints, the ordering discards segment extent; a query region defined over midpoints can therefore exclude a segment whose midpoint lies outside the region while part of that segment lies inside it. The fetch radius must accordingly be inflated by half the global maximum entry extent. It cannot be inflated by the longest segment found within the radius, because that quantity is known only after the fetch it is intended to bound. Entry-extent capping earns its cost at this point: capping reduces the global maximum from 5,748.24 m to 25 m and the inflation margin from 2,874.12 m to 12.50 m, a factor of 229.9 in radius. The measured effect on admitted work exceeds what the radius ratio alone suggests, as the uncapped index admits an average of 5,612.61 midpoints per property against 10.28 for the capped index, a reduction by a factor of 546.

The query proceeds in four phases. First, the property's coordinates are mapped to a Hilbert key and an array position for that key is obtained, either by binary search or by model inference. Second, a fixed window of 64 entries on either side of that position, clamped at the array ends, is scanned with the exact point-to-segment kernel, and the minimum over those real segments is recorded as d_{seed}. The window is not extended, no boundary is probed, and no directional search is performed; the scanning loop has fixed bounds and terminates unconditionally. Third, in what is termed the resolve descent, a disk of radius $R_{seed} = d_{seed} + L/2 + \tau$, where L denotes the capped maximum extent and τ a tie tolerance, is decomposed into Hilbert key ranges and every entry within is scanned exactly, producing the true minimum d_{best}. Fourth, in the final descent, the candidates accumulated by the resolve descent are discarded and a second descent is executed at the tight radius

$R = d_{best} + L/2 + \tau$, which produces the emitted candidate set and the reported counters.

Exactness does not depend upon the window containing the nearest segment. Because d_{seed} is a minimum taken over real geometry, $d_{seed} \geq d_{best}$ holds unconditionally; a poor seed therefore yields a larger d_{seed}, hence a larger disk, and the disk is defined in space rather than in array positions. Violating the guarantee would require $d_{seed} < d_{best}$, which is impossible for a minimum over the same geometry the descent subsequently searches. This condition is not a corner case: under the binary seeder the window fails to contain the nearest segment for 38.62 percent of properties, and results are exact for all of them. Because the final descent's radius is a function of d_{best} alone, its counters are seed-independent by construction, and measurement confirms this; both seeders report exactly 47.5926 entries and 60.3199 nodes per property in that descent. In total, the window sizes the search and the descent performs it.

The acceptance criterion for this argument was declared before the learned seeder existed. A seed mode is provided that forces the seed position to array index 0 for every query, thereby guaranteeing the worst available starting bound; output produced under that mode is byte-identical to output produced by the production configuration.

The region predicate was likewise selected against a declared threshold. An earlier implementation fetched the bounding box of the disk rather than the disk itself, and a gate declared in advance that the box would be replaced if its over-covering at the 99th percentile exceeded a factor of 3. Measured over-covering was a factor of 181 at that percentile; adopting the disk predicate subsequently reduced entries decomposed per property from 89.23 to 47.59.

F. Recursive Model Index

The fifth rung differs from the fourth in exactly one respect: the method by which the seed position is obtained. A two-stage recursive model index replaces binary search. A root linear model selects one of 131,072 leaves, of which 12,449 are occupied under this key distribution, and a leaf linear model predicts a position within the key array. The structure occupies 4,194,400 bytes against a key array of 9,516,712 bytes, thereby satisfying a budget declared in advance at one half of the key array.

Training is deterministic. Model parameters are obtained by ordinary least squares in NumPy at a fixed seed, and refitting reproduces the parameter array byte for byte. The C++ component performs inference only and mirrors the Python trainer including its two rounding orders, since a single differing rounding order would produce a different predicted position and, through the seed window, a different quantity of work. The model is pinned to the index build by two digests, one for the training array and one for the index itself; a model trained against one key array therefore cannot be silently applied to another.

The model's error bound is measured exhaustively over all 1,189,589 keys rather than estimated from a sample. Independently, predicted positions were compared against `std::lower_bound` for all 267,362 property keys. Binary search returns the exact position for every property, as required. The model returns the exact position for 99,453 properties and a position within the 64-entry window for 231,617 properties, or 86.63 percent; its mean absolute position error is 55.77 entries, with a median of 5, a 99th percentile of 731, and a maximum of 17,995. The shape of this distribution matters more than its mean, as the prediction is usually excellent and occasionally catastrophic, and a window of fixed width cannot adapt to that behavior.

Both rungs are additionally characterized by an analytic probe metric adopted from the learned-index literature, $\lceil \log_2(e_{max} - e_{min} + 1) \rceil$, weighted across leaves by key load. The metric counts array halvings and is therefore expressed in the same unit for both designs. Binary search halves 1,189,589 keys, yielding 20.2376, whereas the model halves only the measured error window of its leaf, yielding 6.3225; an equidepth partition of the same size yields 2.97 for reference. It must be noted that the query executes neither procedure. The model performs one multiply-add and no probes whatsoever, and the query reads the fixed window without loading an error bound at all. The metric thus describes a lookup this query does not perform, and Section VII examines the consequences.

G. Terrain Evidence

Terrain data is surveyed via a cached United States Geological Survey 3DEP one-third arc-second DEM. The source raster is reprojected to EPSG:26918 from EPSG:4269, facilitating metric coordinates for local sampling windows, slope calculation, and neighborhood statistics. The reprojected raster is stored with an approximately 8.601 m pixel size. For each property centroid, elevation is sampled; a local mean elevation is computed for a 90 m enclosing raster area; relative elevation is computed as the difference between property elevation and local mean elevation; and slope is computed in degrees using a 3-by-3 Horn-style finite-difference operator. The tabulated terrain output records `property_id`, `terrain_elevation_m`, `terrain_local_mean_elevation_m`, `terrain_relative_elevation_m`, `terrain_slope_degrees`, `terrain_sample_radius_m`, and `terrain_crs`. Terrain evidence is maintained separately from the FEMA/hydrography evidence table for independent inspections of its raster source, checksum, CRS and sampling method.

H. Preliminary Exposure Index

After extracting all evidence fields, CAPRM-Flood builds a preliminary exposure-index table from the FEMA/hydrography and terrain evidence tables. Python performs scoring on this table, under the policy version `preliminary_exposure_index_v2`; it produces normalized component scores on a 0–100 scale for FEMA

evidence, nearest-water evidence, absolute elevation, and relative elevation.

Extracting the FEMA component comprises an absolute lookup rather than a percentile ranking. Absolute values declared for each zone are Zone X at 10.0, Zone AO at 80.0, Zone A at 90.0, Zone AE at 95.0, and Zone VE at 100.0; an unrecognized zone label raises an error rather than defaulting to any value, so an unmapped input cannot silently receive a score. The nearest-water component presents a percentile-based score; shorter nearest-water distances correspond to higher relative exposure. The two terrain components present percentile-based scores separately for low absolute elevation and for low relative elevation. Slope is extracted and retained as terrain evidence but does not contribute to any component score. Finally, the preliminary exposure index is computed as a weighted sum, for which current weights are 0.40 for the FEMA component, 0.35 for the nearest-water component, 0.15 for the absolute elevation component, and 0.10 for the relative elevation component. The composite is rounded to nine decimal places before ranking, so that percentile assignment is a deterministic function of the recorded index values rather than of floating-point residue below the recorded precision. The final index table records `property_id`, the four component scores, `exposure_index_0_100`, `exposure_percentile`, and `scoring_policy_version`.

The four weights are held in a single flat, recorded set rather than distributed across nested policy structures. This arrangement satisfies a design requirement rather than a stylistic preference: any scoring constant absent from the recorded manifest breaks reproducibility, because the recorded configuration must by itself suffice to recompute the recorded output. A flat weight set makes the completeness of that record inspectable.

The scoring stage is intentionally implemented after the lower-level evidence stages rather than inside them, for independent FEMA, water, and terrain evidence analysis before preliminary relative rankings. The resulting index supports countywide screening and comparison.

I. Coordinate Surrogate

The surrogate maps projected coordinates to an exposure score. Its input is a Gaussian random Fourier feature encoding of the normalized coordinate pair, comprising 64 frequencies at scale 256 and corresponding to a characteristic wavelength of 99.32 m, with the raw coordinates retained; this encoding is followed by a two-layer network of width 128. The surrogate is trained against the pipeline's own deterministic output, so the label contains no measurement noise and any observed error is therefore a property of the model rather than of the data.

The input comprises coordinates only. No FEMA zone, elevation, or water distance is supplied. This restriction is deliberate rather than incidental: the exposure index is a fixed weighted sum of four component scores, so a model given those components would be fitting a linear function of its own inputs and would establish nothing about whether the field is representable from position. The restriction is thus what makes the study a question about spatial structure, and it is likewise the reason the result reported in Section VIII takes the form it does.

The recursive model index and the coordinate surrogate are unrelated systems, and the two are easily conflated because both are learned. The former maps a key to an array position, remains exact, and requires no partitioning of data; the latter maps a position to a score, is approximate by design, and requires the spatial partitioning described in Section VIII.

J. Reproducibility Artifacts

CAPRM-Flood outputs both intermediate and final artifacts with explicitly-defined paths, schemas, CRS, and validation summaries. Python is used to generate baseline outputs, C++ inputs, comparison against C++ results, benchmark summaries, evidence and index tables, and JSON manifests. The JSON manifests record input paths, I/O checksums, row counts, schema information, CRS metadata, reference implementation benchmarks, and aggregated evidence statistics. Central CAPRM-Flood artifacts include the property cache, Python FEMA baseline, Python nearest-water reference, C++ FEMA output, C++ brute-force nearest-water output, C++ indexed nearest-water output, benchmark records, FEMA and Hydrography evidence table, terrain evidence table, and exposure-index table.

Because the implemented artifact structure records the source and validation context needed to determine how the workload files were produced, each major transformation to the workload becomes auditable, and cached sources can be used for repeatable execution.

V. Current Validation and Results

CAPRM-Flood currently validates its vector spatial computations by comparing independently computed C++ outputs against outputs computed via GeoPandas/Shapely reference implementation in Python. Property ID matches points across code environments; a full correctness validation requires complete row coverage and categorical agreement across source-feature identifier, CRS, and nearest-water distance agreement within a tolerance of 1e-6 m. FEMA validation verifies across both implementation outputs the polygon-match status, SFHA status, flood-zone label, and FEMA feature index. Nearest-water validation compares distance, as well as the identifier, class and type corresponding to the nearest hydrography feature, and the source identifiers, feature name, tie count, and distance CRS for each point.

The countywide evidence table contains one row for each Monroe County property identifier sampled. From the sampled properties, 267,358 matched FEMA flood-hazard polygons and four did not. FEMA classification data contained 5,061 SFHA properties and 262,301 non-SFHA properties; observed FEMA zone counts included 262,297 Zone X properties, 4,226 Zone AE properties, 408 Zone A properties, 388 Zone AO properties, 39 Zone VE properties, and four unmatched

TABLE I
COUNTYWIDE VALIDATION SUMMARY

Metric	Countywide result
Property identifiers	267,362
FEMA Python/C++ agreement	267,362/267,362
Brute-force water agreement	267,362/267,362
Feature-hierarchy water agreement	267,362/267,362
Segment-hierarchy water agreement	267,362/267,362
Hilbert binary-seed water agreement	267,362/267,362
Hilbert learned-seed water agreement	267,362/267,362
Water fields compared per rung	10
Coverage basis	union of ID sets
Maximum water distance error	9.157×10^{-10} m
Composite recomputation difference	5.00×10^{-10}
Python test suite	653/653 passing
Artifact audit	49 pass, 1 warn, 0 fail

properties. Regarding nearest-water evidence, 266 properties had zero distance to a retained hydrography feature, 308 properties had multiple equally nearest features, and 2,934 unique hydrography features were selected as the nearest water feature for at least one property. Median nearest-water distance was 325.346 m, mean distance was 506.411 m, and maximum distance was 2,630.235 m.

Terrain evidence comprises countywide elevation, local mean elevation, relative elevation, slope, sampling radius, and terrain CRS fields for all 267,362 property identifiers. Elevation ranged from 75.000–296.309 m, with mean value 143.197 m and median value 146.828 m. Relative elevation values ranged from -20.669–30.149 m, with mean 0.247 m and median 0.175 m. Mean slope was 1.906 degrees, median slope was 1.234 degrees, and there were no missing countywide slope values; all terrain evidence was reported in EPSG:26918.

FEMA, nearest-water, and terrain components are combined in the preliminary exposure-index stage under policy version `preliminary_exposure_index_v2`. Implemented weights are 0.40 for the FEMA component, 0.35 for the nearest-water component, 0.15 for the absolute elevation component, and 0.10 for the relative elevation component. The resulting countywide exposure index ranged from 7.915 to 99.929, with a mean of 34.632 and median of 33.728; corresponding countywide exposure percentiles were also recorded. An independent recomputation of the composite from the recorded component columns and the recorded weights agrees with the stored index to a maximum absolute difference of 5.00×10^{-10}, and ranking the stored index reproduces the stored percentile to a maximum absolute difference of 5.01×10^{-13}.

A. Cross-Implementation Agreement Across Five Index Designs

Each rung of the indexing ladder was compared against the frozen Python reference on ten fields of the nearest-water evidence record: the computed distance, the identifier of the selected feature, that feature's class and type, its source identifier and source object identifier, its name, the count of features tied at the minimum, the distance CRS, and a conjunction across all preceding fields. Every rung agrees with the reference on all ten fields for all 267,362 properties. Distance is compared under a tolerance of 10^{-6} m; the observed maximum absolute error is 4.658×10^{-10} m for the brute-force and hierarchical rungs and 9.157×10^{-10} m for the Hilbert rungs, four orders of magnitude within the applied tolerance. All remaining fields are compared under exact equality.

Two properties of this comparison bear on what the agreement establishes. First, the tie count is compared rather than only the selected feature; two implementations could agree on both distance and selected feature while disagreeing as to how many features were tied at that minimum, which would indicate a difference in tie-breaking behavior that a distance comparison alone would not detect. Second, coverage is computed over the union of the two property-identifier sets rather than over an inner join, so a rung silently dropping rows would fail coverage rather than score perfectly across the rows it retained. Coverage is 1.0 for every rung, with zero rows missing on either side.

Each rung is validated independently against the Python reference rather than against the rung preceding it. The ladder thus constitutes five separate verifications sharing a single reference, rather than a chain in which an error introduced at one rung could propagate upward undetected.

B. Test Suite and Artifact Audit

The Python test suite comprises 653 tests, all of which pass. A separate structural audit inspects the stored evidence, terrain, and index artifacts rather than the code producing them, verifying manifest path and checksum agreement, schema and column presence, null counts, value ranges, property population identity across the three tables, derivation identities, deterministic row ordering, and policy version consistency. The audit returns 49 passes, 1 warning, and 0 failures. The warning records that two manifest key conventions are currently in use across the products, such that any tool reading manifests generically must handle both. An audit pass establishes that the artifacts are internally consistent and describe an identical property population; it does not establish that the underlying evidence is accurate or that the scoring methodology is correct.

VI. SENSITIVITY ANALYSIS

The exposure index combines four component scores under fixed weights. Because the weights are selected rather than derived, the degree to which the resulting ranking depends upon that selection is itself a measurable property of the system.

A. Perturbation Design

Weight perturbations are drawn from a Dirichlet distribution over the simplex with a concentration parameter of 200 times the baseline weights, at a fixed seed. Two properties follow from this choice. Every draw is a valid weight vector by construction, with non-negative components summing to one, so no draw requires discarding or renormalization. Further, the draws are centered on the baseline rather than selected by

hand, so the resulting spread describes plausible disagreement regarding the weights rather than a set of alternatives chosen after observing the outcome.

Perturbations alone cannot be interpreted. If a family of near-baseline weightings yields a rank correlation of 0.9, that value carries no meaning absent knowledge of what a substantially different weighting yields. Four single-component reference corners, each placing all weight on one component, are therefore evaluated alongside the 36 plausible configurations, for 40 scenarios in total. The corners yield a minimum rank correlation of 0.1674 and a minimum top-decile overlap of 0.2895, which establishes the scale against which the perturbation results are subsequently read.

Thresholds were declared prior to measurement. A result is classified as stable if the minimum Spearman correlation across scenarios is at least 0.95 and the minimum top-decile overlap is at least 0.80, and as moderately sensitive if those minima are at least 0.85 and 0.60 respectively.

B. Measured Stability

The classifier returns moderately sensitive. The minimum Spearman rank correlation against the baseline ranking is 0.8750, attained by the equal-weight scenario, against a median of 0.9963. The minimum top-decile overlap is 0.7614 against a median of 0.9455. The largest percentile shift observed for any property under any plausible scenario is 47.47 points, whereas the median across scenarios of the median per-property shift is 1.56.

Reporting the verdict as the pre-declared classifier returned it is more important than reporting the medians. The medians in isolation would support a claim of stability that the thresholds established in advance by this project do not.

C. Nominal Weight and Measured Influence

Variance shares are computed by covariance decomposition, as $\mathrm{Cov}(w_i C_i, I)/\mathrm{Var}(I)$, which sums to exactly one by linearity of covariance and therefore does not assume the components are uncorrelated. That assumption would not hold in any case, as the largest pairwise rank correlation between components is 0.1522, which is small but nonzero.

The decomposition and the nominal weights diverge substantially. Water proximity carries 35 percent of the weight and 65.3 percent of the variance in the composite, whereas the FEMA component carries 40 percent of the weight and 16.8 percent of the variance. The mechanism is visible in the component distribution: the FEMA component takes only six distinct values across the workload, and 262,297 of 267,362 properties, or 98.11 percent, share the same one. Adding a constant to 98.11 percent of a set does not change the ranking of those members relative to one another, and the exposure index is interpreted as a ranking.

This behavior is a property of the design rather than a defect within it. The FEMA component's role is not to spread the population but to displace a small subset decisively. The interval between the lowest and highest mapped zone values is 85 points, so a property crossing from the Zone X value to Zone VE is displaced by $0.40 \times 85 = 34$ points on a composite whose standard deviation is 13.06, or approximately 2.6 standard deviations. In total, a component can be simultaneously the largest nominal weight and a minor contributor to variance; reporting only the weights would therefore misdescribe what drives the ranking.

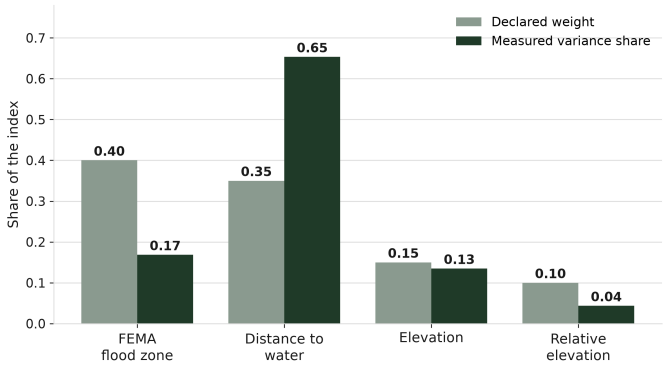

Fig. 1. Nominal weight against measured variance share for the four scoring components. Shares are computed as $\mathrm{Cov}(w_i C_i, I)/\mathrm{Var}(I)$ and sum to exactly one by linearity of covariance; no assumption of independence between components is made.

VII. Performance Evaluation

A. Measurement Protocol

Each configuration is executed repeatedly and the median is reported. Repetition counts differ by rung and are stated rather than averaged over: the brute-force countywide run was executed once, the segment-level hierarchy seven times, and the seed-window sweep seven times per cell with warm-up invocations discarded. A single-run figure cannot carry an interval and is marked accordingly.

A comparison between two configurations is reported as resolved only if the interval $[\min_a / \max_b,\ \max_a / \min_b]$, formed from the observed per-repetition extremes, excludes 1.0. The rule makes no distributional assumption and requires no appeal to a standard error. It was applied uniformly, including to comparisons where it withholds a result that the medians in isolation would have supported.

Timings obtained from different harness invocations of an identical configuration differ by as much as approximately ten percent on this machine. Every figure in Table II is accordingly drawn from a single invocation, and figures from other invocations are not mixed into it. Within-cell spread also differs by invocation. The seed-window sweep carries wider per-repetition ranges than the dedicated ladder runs, with the consequence that a difference resolved in one invocation may be unresolved in another at the same nominal repetition count. Resolution verdicts are accordingly reported per invocation rather than pooled.

B. Indexing Ladder Results

Table II reports all five rungs at the shipped operating point: a 25 m entry-extent cap where a cap applies, the disk

TABLE II
INDEXING LADDER, COUNTYWIDE WORKLOAD, ORIGINAL VERIFICATION, SINGLE HARNESS INVOCATION

Rung	Design	Checks/prop.	μs/prop.	Speedup
1	Brute force	1,063,159	3,993.9	1.0×
2	Feature hierarchy	70,771	235.4	17.0×
3	Segment hierarchy	9,408	34.1	117.1×
4	Hilbert, binary seed	11,022	66.4	60.2×
5	Hilbert, learned seed	11,022	70.7	56.5×

region predicate, original verification, and a seed window of 64 entries.

The ladder is not monotone. Computation time falls by two orders of magnitude across the first three rungs and then rises, such that both one-dimensional rungs are approximately twice as slow as the segment-level hierarchy. The same ordering holds under split verification, in which the segment-level hierarchy runs at 6.1 microseconds per property against 35.5 and 39.9 for the two Hilbert rungs; the result is therefore not an artifact of how candidates are verified.

C. Search Cost Versus Verification Cost

Flattening two dimensions to one costs 17.2 percent additional exact segment checks per property, at 11,022 against 9,408 under original verification and 7,617 against 6,454 under split verification. Both rungs are constructed from the same 1,189,589 capped entries, so the cap cannot account for the difference and the comparison isolates dimensionality. A 17 percent increase in checks does not, however, account for a twofold increase in time.

Decomposing query time into search and verification does account for it. The segment-level hierarchy spends 0.216 microseconds per property locating candidates and 35.788 microseconds verifying them, whereas the Hilbert rung under a binary seed spends 28.433 microseconds searching and 41.929 microseconds verifying. Search costs approximately 132 times more under the ordering than under the hierarchy, and that term rather than the check count is where the time is spent. The hierarchy rejects entire subtrees against a single bound and narrows to 1.4662 candidate features per property, whereas the ordering must decompose a disk into key intervals and scan every entry within two successive descents, comprising 141.17 entries in the resolve descent and 47.59 in the final descent, in addition to the 128 entries of the fixed seed window.

The search share attributed to the hierarchy is not measured directly. It is assigned from operation counts, comprising approximately 35 box and node operations per property against 9,408 segment checks, or approximately 0.4 percent, with an independent estimate near 0.6 percent; the larger figure is used so that the verification term is not overstated. The Hilbert search figures are measured directly.

D. Probe Reduction Without Time Reduction

The learned seeder reduces the analytic probe metric from 20.2376 to 6.3225, a factor of 3.20, and is nevertheless slower. At a 64-entry window under split verification the learned seeder requires a median 11.00 s against 9.73 s, a ratio of 1.1313 whose interval $[1.0590,\ 1.2036]$ excludes unity at seven repetitions per cell; the difference is therefore resolved. Expressed per property, the gap is 4.24 to 5.08 microseconds.

The mechanism is a chain of measured quantities, and it does not involve a longer search. The predicted position is worse, at a mean absolute error of 55.77 entries against zero for binary search, so the fixed window is centered on less relevant geometry and fails to contain the nearest segment for 46.01 percent of properties against 38.62 percent. The seed bound is correspondingly looser, as the mean ratio d_{seed}/d_{best} is 1.5388 against 1.1717, with a maximum of 517.3 against 32.2. A looser bound produces a wider disk, and a wider disk admits more entries: 352.52 per property in the resolve descent against 141.17, a difference of 211.34.

That difference accounts for the observed timing gap. The marginal cost of one resolve-descent entry was measured in isolation, prior to any of these timings, at 20.4 ns. At that rate the predicted penalty is $211.34 \times 20.4\ \text{ns} = 4.31$ microseconds per property, against a measured 4.24 to 5.08 microseconds. Fitting the constant post hoc across all 29 matched configuration pairs returns 20.50 ns, which agrees with the value declared in advance.

E. Dependence on Seed Window Size

The seed window is a compile-time constant rather than a runtime flag, because a runtime branch would alter loop unrolling within the timed region and thereby change what is being measured.

Sweeping the window from 8 to 2,048 entries demonstrates that the two seeders do not maintain a fixed relationship. Both improve as the window widens: the binary seeder's resolve descent falls from 167.97 entries per property at a window of 8 to 77.46 at 2,048, and the learned seeder's from 523.04 to 94.19. The learned seeder improves more rapidly because it begins from a worse bound and therefore has more to recover.

The clearest single statement of the trade is an exchange rate. The learned seeder saves a constant 20.238 probes per property at every window size, since it performs one multiply-add and no probes at all, while the additional resolve entries it costs fall as the window widens. The resulting price, expressed as additional entries paid per probe saved, is 10.44 at a window of 64 and 0.83 at 2,048. Whether the trade is favorable is therefore not a property of the two index designs but a property of a configuration parameter, and it moves by more than an order of magnitude across the range within which that parameter is plausibly set.

Applying the resolution rule to every cell of the sweep separates what the measurements support from what they merely suggest. Under split verification the learned seeder is resolvably slower at every window size from 8 through 256 entries, with intervals excluding unity in all six cells, and ceases to be distinguishable from the binary seeder at 512 entries and beyond. Under original verification only the smallest window yields a resolved difference, so that mode's

sweep supports the direction of the effect but not its magnitude at any particular window.

The apparent reversal is not resolved and is not claimed. Under split verification the median ratio falls below unity at a window of 1,024, reaching 0.9953, but its interval $[0.9654,\ 1.0340]$ contains unity, as does the interval at 2,048 entries. Under original verification the ratio converges to 1.0000 at 2,048 entries without falling below it. What the sweep establishes is therefore convergence to parity rather than a change of rank. Two further observations bear on how that parity should be read. Both time curves are U-shaped, such that at a window of 1,024 the binary seeder requires 11.08 s against 9.68 s at its own optimum of 128 entries; parity is thus reached within a regime approximately 14 percent worse than the best configuration measured, for both designs. Further, parity is reached by the penalty vanishing into measurement noise rather than by the learned seeder becoming faster.

The learned-index literature reports probe counts and does not report this dependence. Over this workload the probe metric predicts neither the size of the penalty nor the window size at which it ceases to be measurable.

VIII. Surrogate Feasibility

This study asks whether the exposure index can be approximated from coordinates alone. The answer is negative, and that negative answer constitutes the result. What makes it a result rather than a failure is that the evaluation was constructed to be capable of returning the opposite: a partitioning strategy whose geometry is justified against measured spatial structure, a performance floor declared prior to training, a predictive claim registered prior to training, and a verdict procedure carrying positive controls for both outcomes.

A. Constructing a Partition That Does Not Leak

The median distance from a property to its nearest neighboring property is 24.39 m, and 1,700 rows are separated by exactly zero distance. A random partition therefore places most held-out rows within a few tens of meters of a training row.

Whether that condition constitutes leakage depends upon the correlation length of the field, which CAPRM-Flood estimates from the workload itself. Two independent passes over the empirical semivariogram, the first using sampled ball queries against all points at short lags and the second using exhaustive pairwise computation over a random subsample at long lags, place the lag at which the semivariance reaches half the sample variance at 2,125 m and 2,137.5 m respectively. The field's dependence thus extends approximately two orders of magnitude beyond the distance a random partition leaves between a held-out row and its nearest training row.

That quantity is model-free, and it is the quantity against which the block geometry is chosen, for a reason worth recording. Fitting an exponential variogram model to identical data returns a range parameter moving from 503.5 m to 16,592.5 m depending solely upon the maximum lag included in the fit, while the coefficient of determination remains above 0.95 throughout. A value moving by a factor of thirty in response to an unreported choice cannot support a design decision; it is therefore reported as a swept diagnostic and nothing is built upon it.

TABLE III
Blocked K-Fold Root-Mean-Square Error Across Five Seeds

Predictor	RMSE range
Constant (training mean)	13.39 – 15.92
Nearest training neighbor	14.05 – 16.53
Coordinate surrogate	12.30 – 16.06

The evaluation accordingly uses blocked K-fold cross-validation with 10,000 m blocks, $K = 5$, five seeds, a buffer of 2,125 m, and a test partition isolated from the validation partition as well as from the training partition. A separation gate confirms the construction across all 25 fold-seed cells, and both controls, comprising an unbuffered blocked partition and a random partition, fail that gate.

One alternative explanation is excluded by measurement. Co-located properties could impose an irreducible error floor by carrying differing labels at identical coordinates. Across 266,038 distinct coordinates there are 376 groups holding more than one property, covering 1,700 rows, and none contains a differing label; the irreducible root-mean-square error is 6.3×10^{-16}. The floor is therefore not attributable to label noise.

B. Surrogate Performance Under the Blocked Partition

The performance floor declared at partition construction is a nearest-training-neighbor predictor. Under the blocked partition it achieves a root-mean-square error between 14.05 and 16.53 across seeds. The surrogate achieves between 12.30 and 16.06, beating the floor at four of five seeds and losing at one.

Reporting that comparison in isolation would be misleading, and CAPRM-Flood adds a third reference point demonstrating why. The declared floor's coefficient of determination is negative at every seed, which means the floor is beaten by predicting the training mean. Predicting a constant achieves between 13.39 and 15.92. All three ranges overlap, as shown in Table III.

Under a partition constructed to resist leakage, the surrogate is not separable from a constant predictor. Clearing the declared floor is therefore not by itself evidence that spatial structure was learned, and the reference point demonstrating this is reported alongside the floor rather than left for a reader to derive.

Two further observations follow. The spread across seeds is 3.76 under the blocked partition against 0.14 under the random control, so a single-seed result would be uninformative and five seeds constitute a requirement rather than diligence. Further, the same nearest-neighbor predictor evaluated under the random partition achieves 4.93 with a coefficient of determination of 0.859, against 14.05 to 16.53 and -0.54 to -0.15 under the blocked partition. The two rows describe a single predictor evaluated two ways; the difference between them is therefore the magnitude of the error a random partition would have introduced, rather than a property of any model.

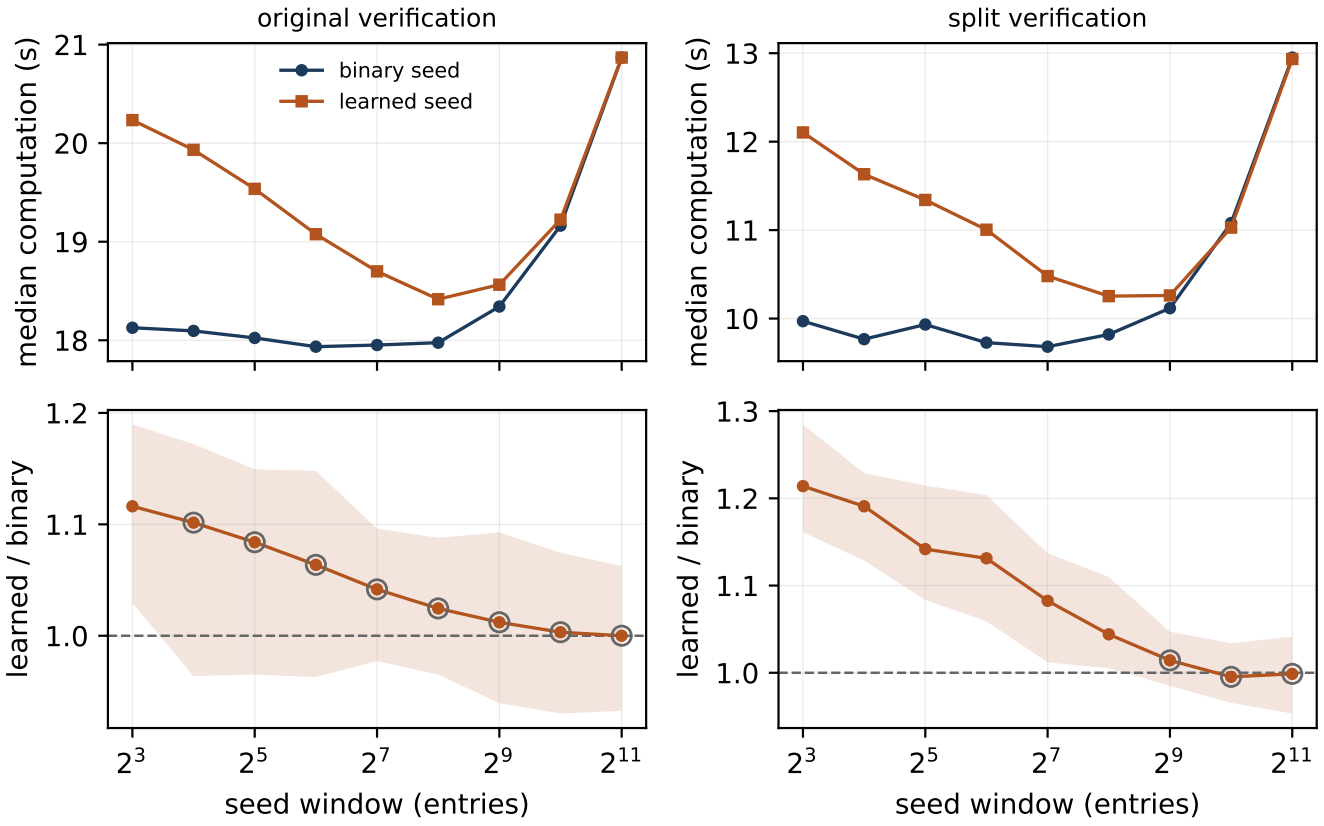

Fig. 2. Seed-window sweep, countywide workload, seven repetitions per cell with warm-up invocations discarded. Top row: median computation time against seed window size. Bottom row: ratio of learned-seed to binary-seed time, with the shaded band giving the interval $[\min_{\mathrm{rmi}} / \max_{\mathrm{bin}},\ \max_{\mathrm{rmi}} / \min_{\mathrm{bin}}]$ and hollow rings marking cells whose interval contains unity and which are therefore not resolved.

Training is deterministic. Refitting one fold reproduces the weight digest byte for byte and the resulting error to twelve decimal places.

C. A Registered Prediction and Its Refutation

Prior to training, the following claim was recorded: because the FEMA component is a discontinuity and 98.1 percent of properties sit at an identical value, and because a smooth network can represent a ramp but not a step, residuals should spike along zone boundaries and remain small elsewhere. The claim was recorded as not subject to subsequent adjustment.

The claim contains two separable assertions. The first, that error concentrates within the minority classes, holds. Under the blocked partition the minority root-mean-square error is 45.13 against 13.29 for the majority, and 1.93 percent of rows carry 18.54 percent of the total squared error.

The second, that error is spatially organized around zone boundaries, fails, and it fails differently under the two partitions, which is what establishes that the test was capable of confirming. A threshold of $|\rho| \geq 0.10$ was declared for the correlation between residual magnitude and distance to the nearest property of a differing class, computed within label strata. Under the blocked partition the correlation clears the threshold at $+0.112$ but carries the wrong sign, as error rises with distance from a boundary rather than toward it. Under the random control the sign is correct at -0.057 but the magnitude does not clear the threshold. Neither partition can therefore produce a confirmation, and each fails a different one of the two conditions. The verdict is refuted.

A direct measurement of how much of the step survives supports the same conclusion. Binning by label and comparing the span of the labels against the span of the predictions across those bins, the labels span 25.73 points and the predictions span 3.17, for a recovery ratio of 0.123. The model thus reproduces approximately one eighth of the structure it was asked to represent.

The boundary measure is a proxy and is reported as one, with three declared limitations. It is bounded below by property spacing, so a parcel on a polygon edge does not read zero; its observed minimum is 4.55 m and its median is 966.34 m. It locates a boundary only as precisely as parcels happen to sample it. Finally, an SFHA polygon containing no parcels is invisible to it. Four competing explanations for the residual pattern were evaluated by the same estimator, with uncertainty obtained from a cluster bootstrap resampling whole properties rather than rows; the strongest competitor differs between the blocked partition and the random control, which is itself

evidence that the random control measures something other than what the blocked partition measures.

D. Inference Cost

Inference cost was measured under a boundary declared in source prior to any run: warm caches only, with cold start explicitly not measured and not claimed; setup and compute reported as separate columns, because the shipped command-line tool digests a 21.2 MB package on every invocation and that cost belongs to the tool rather than to the algorithm; and marginal cost obtained from a linear fit across three workload sizes, reported with its residual, with three points acknowledged as the minimum supporting a linearity claim rather than a comfortable number of them.

The exact path costs 35.9 microseconds per property amortized across the countywide batch. The surrogate costs 2.28 microseconds at one thread and 1.98 microseconds at eight threads when batched, or approximately sixteen to eighteen times less. At batch size one it costs 17.71 microseconds, and the speedup obtained from eight threads at that batch size is 0.997 with an interval of $[0.973, 1.049]$, which the resolution rule reports as unresolved. The advantage is thus a batching advantage rather than a latency advantage.

The internal distribution of cost explains why threading does not assist. Feature encoding costs 1.54 microseconds at one thread against 0.98 microseconds for the network itself, and the network parallelizes at a factor of 2.02 while the encoding achieves only 1.04; the dominant stage is therefore the stage that does not scale. Agreement against the reference implementation reaches a maximum deviation of 1.87×10^{-5} against a declared tolerance of 3.87×10^{-6}, breached only at batch size one and identical across thread counts, which identifies the cause as kernel dispatch at that operand shape rather than a change in reduction order.

IX. Limitations and Threats to Validity

The results reported here describe one county, one set of dataset vintages, and one hardware configuration. The relative ordering of index designs is a measurement over this workload on this machine, and the window-size dependence reported in Section VII is precisely a demonstration that such orderings can move with configuration.

Agreement across five implementations validates the implementation rather than the method. Five independent programs reproducing an identical distance to within a nanometer establishes nothing regarding whether distance to the nearest mapped water feature is a suitable proxy for flood exposure.

The evidence products are generated by the brute-force and feature-level hierarchy implementations, which were validated early and subsequently frozen. The segment-level hierarchy is equally exact and approximately seven times faster, so the configuration actually used to produce the evidence table is not the fastest validated configuration available. Adopting it would require regenerating and revalidating every downstream product, which was not undertaken.

The exposure index is a relative regional ranking. It is not a calibrated flood probability, an expected loss, an insurance price, or an actuarial estimate, and it should not be interpreted as any of these. Three of its four components are percentile ranks computed within the workload, so a property's score is a statement regarding its position among these 267,362 properties and is not comparable across workloads or across NFHL vintages. The sensitivity classifier returned moderately sensitive against thresholds selected by this project, and no external standard exists against which to judge those thresholds.

The boundary proxy employed in the surrogate study is bounded below by parcel spacing and cannot detect an SFHA polygon containing no parcels. The surrogate result is conditional upon coordinates-only inputs and does not bear upon what a model supplied with the components could achieve, which would constitute a different and considerably less informative question.

The decomposition of query time into search and verification assigns the hierarchy's search cost from operation counts rather than measuring it directly. The artifact audit returns one warning, recording that two manifest key conventions coexist across the products. Finally, the canonical project documentation currently lags the implementation.

X. Conclusion

Both clauses of the research question admit direct answers.

Exact agreement with a trusted Python baseline is preserved across the pipeline and across every index design evaluated. Five independent C++ implementations, spanning brute force, two hierarchy granularities, and a space-filling-curve ordering seeded two different ways, reproduce the reference on all ten fields of the nearest-water evidence record for all 267,362 properties, within 10^{-9} m on distance and exactly on every identifier, name, and tie count.

Regarding reductions in computation time across strategies, exact segment-level hierarchical indexing presented as the appropriate default for this workload. The one-dimensional alternatives, including the learned approach, did not improve on minimum measured wall-clock time, as a hierarchy's search is nearly free while an ordering's search is not. The learned seeder reduced the analytic probe metric by a factor of three and cost time, as the metric describes a lookup the query does not perform. The ranking between the two seeders is additionally a function of seed window size, which situates the question at a configuration boundary rather than at a settled result.

Two results in this work were produced by tests which returned against the outcome anticipated for them. The registered prediction concerning the spatial organization of surrogate error was refuted, and the resolution rule did not certify the apparent reversal between seeders.

The evaluation harness is reproducible, deterministic, and instrumented, which is the contribution outlasting the specific measurements reported here. The repository permits the ladder to be rerun at a different scale, on different hardware, or at a different window size, in order to establish whether the

convergence observed here is a property of this machine or of the method.

ACKNOWLEDGMENT

The author thanks Professor Minseok Kwon for his guidance and feedback throughout the development of this capstone project.

REFERENCES

[1] National Severe Storms Laboratory, "Severe Weather 101: Flood Basics," National Oceanic and Atmospheric Administration. [Online]. Available: https://www.nssl.noaa.gov/education/svrwx101/floods/. [Accessed: Jun. 9, 2026].

[2] U.S. Global Change Research Program, *Fifth National Climate Assessment*. Washington, DC, USA: U.S. Global Change Research Program, 2023. [Online]. Available: https://toolkit.climate.gov/sites/default/files/2025-07/NCA5_2023_FullReport.pdf. [Accessed: Jun. 9, 2026].

[3] Federal Emergency Management Agency, "One inch of floodwater can cause up to $25,000 of damage in a home." [Online]. Available: https://www.fema.gov/vi/print/pdf/node/633873. [Accessed: Jun. 9, 2026].

[4] Monroe County, New York, *Hazard Mitigation Plan 2023 Update, Volume I*, Mar. 2023. [Online]. Available: Monroe County Hazard Mitigation Plan. [Accessed: Jun. 9, 2026].

[5] Federal Emergency Management Agency, "Flood Data Viewers and Geospatial Data: National Flood Hazard Layer." [Online]. Available: https://www.fema.gov/flood-maps/national-flood-hazard-layer. [Accessed: Jun. 9, 2026].

[6] Federal Emergency Management Agency, "FEMA Flood Map Service Center." [Online]. Available: https://msc.fema.gov/portal/home. [Accessed: Jun. 9, 2026].

[7] New York State GIS Program Office, "Parcels: New York State Parcels." [Online]. Available: https://gis.ny.gov/parcels. [Accessed: Jun. 9, 2026].

[8] GeoPandas Developers, "Merging data," GeoPandas Documentation. [Online]. Available: https://geopandas.org/en/stable/docs/user_guide/mergingdata.html. [Accessed: Jun. 9, 2026].

[9] V. Pandey, A. Kipf, T. Neumann, and A. Kemper, "How good are modern spatial analytics systems?" *Proceedings of the VLDB Endowment*, vol. 11, no. 11, pp. 1661–1673, 2018, doi: 10.14778/3236187.3236213.

[10] GeoPandas Developers, "geopandas.sjoin," GeoPandas Documentation. [Online]. Available: https://geopandas.org/en/v1.1.2/docs/reference/api/geopandas.sjoin.html. [Accessed: Jun. 9, 2026].

[11] Shapely Developers, "Predicates," Shapely Documentation, ver. 2.1.1. [Online]. Available: https://shapely.readthedocs.io/en/2.1.1/predicates.html. [Accessed: Jun. 9, 2026].

[12] Shapely Developers, "STRtree," Shapely Documentation. [Online]. Available: https://shapely.readthedocs.io/en/2.1.1/strtree.html. [Accessed: Jun. 9, 2026].

[13] Federal Emergency Management Agency, "National Risk Index." [Online]. Available: https://www.fema.gov/flood-maps/products-tools/national-risk-index. [Accessed: Jun. 9, 2026].

[14] Federal Emergency Management Agency, "Hazus." [Online]. Available: https://www.fema.gov/flood-maps/products-tools/hazus. [Accessed: Jun. 9, 2026].

[15] Federal Emergency Management Agency, "Flood Assessment Structure Tool." [Online]. Available: https://www.fema.gov/flood-maps/products-tools/hazus. [Accessed: Jun. 9, 2026].

[16] M. F. Abdullah, S. Siraj, and R. E. Hodgett, "An overview of multi-criteria decision analysis (MCDA) application in managing water-related disaster events," *Water*, vol. 13, no. 10, art. no. 1358, 2021, doi: 10.3390/w13101358.

[17] H. Allafta and C. Opp, "GIS-based multi-criteria analysis for flood prone areas mapping in the trans-boundary Shatt Al-Arab basin, Iraq-Iran," *Geomatics, Natural Hazards and Risk*, vol. 12, no. 1, pp. 2087–2116, 2021, doi: 10.1080/19475705.2021.1955755.

[18] S. Chengu, M. Assen, and E. Gebeyehu, "Multi-criteria decision analysis for flood hazard mapping," *Discover Environment*, vol. 3, art. no. 110, 2025, doi: 10.1007/s44274-025-00289-5.

[19] Boost C++ Libraries, "R-tree quick start," Boost.Geometry Documentation. [Online]. Available: https://www.boost.org/doc/libs/latest/libs/geometry/doc/html/geometry/spatial_indexes/rtree_quickstart.html [Accessed: Jun. 9, 2026].

[20] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. 2018 Int. Conf. Management of Data (SIGMOD '18)*, Houston, TX, USA, 2018, pp. 489–504, doi: 10.1145/3183713.3196909.

[21] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz, "Analysis of the clustering properties of the Hilbert space-filling curve," *IEEE Trans. Knowl. Data Eng.*, vol. 13, no. 1, pp. 124–141, Jan./Feb. 2001, doi: 10.1109/69.908985.

[22] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi: 10.1111/ecog.02881.

[23] M. Tancik *et al.*, "Fourier features let networks learn high frequency functions in low dimensional domains," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 7537–7547.